Sheet 5

Objective: upon successful completion of this sheet, students should be able to effectively apply one of the fundamental OOP principles: inheritance. This encompasses concepts such as method overriding, method overloading, and polymorphism.

1. Design a class named "`Point2D`" that contains the following data members and methods.

   Data members: two private instance variables x (`double`) and y (`double`) to represent an $x-$ and a $y-$coordinate.

   Methods: suitable constructor(s), setters, getters, `setXY()` to set both coordinates, `getXY()` to retrieve both coordinates as an array, `distance(Point2D)` to calculate the Euclidean distance from this point to the specified point, and `distance(double x, double y)` to calculate the Euclidean distance from this point to the specified coordinates.

   The `equals()` and `toString()` methods should be overridden. The former is used for comparing two points based on their $x-$ and $y-$coordinates, while the latter is used to display the point instance in the following format: (x, y).

   - Design also a class named "`Point3D`", which is a subclass of the `Point2D` class and contains the following.

     Data members: private instance variable z(`double`) to represent a $z-$coordinate.

     Methods: suitable constructor(s), setters, getters, `setXYZ()` to set all the three coordinates, `getXYZ()` to retrieve the three coordinates as an array, `distance(Point3D)` to calculate the Euclidean distance from this point to the specified point, and `distance(double x, double y, double z)` to calculate the Euclidean distance from this point to the specified coordinates.

     The `equals()` and `toString()` methods should also be overridden.

   - Test the classes with an array of 5 2D points, two of them are created as 3D points. Then calculate the distances between the 2D points as well as between the 3D points. Check whether the 2D points are equal. Repeat the same checking process for the 3D points.

2. Design a class named "`Figure`" which contains the following.

   Data members: two private instance variables, `startingPoint(Point2D)` to represent the starting point of the figure, and `color(String)` to represent the color of the figure.

   Methods: suitable constructor(s), setters, getters, and a `display()` method to show the figure's information.

   - Design also a class named "`ClosedFigure`" which is a subclass of the `Figure` class and contains the following.

   Data members: three instance variables, `width(float)`, `height(float)` to represent the width and height of the figure, and `filled(Boolean)` to indicate whether the figure is filled with the specified color.

   Methods: suitable constructor(s), setters, getters, and an `area()` method. The `display()` method should be overridden.

   - Design also a class named "`Rectangle`" which is a subclass of the `ClosedFigure` class and contains the following.

   Methods:

   - suitable constructor(s), one of them takes an ending point of type `Point2D` as an argument. Then, calculate the width and height of the rectangle accordingly.
   - The `area()` and `display()` methods should be overridden. The `display()` method shows the rectangle's information, including its area.
   - Test the classes by defining objects from the three class types and checking the methods for each one of them. Declare objects from the `Figure` class, where some of these objects are created as `ClosedFigure` objects and others as `Rectangle` objects. Check the methods for these different types of objects.

3. Design a class named "`Vehicle`" that contains the following data members and methods.

   Data members: `name(String)`, `color(String)`, `price(double)`, `numberOfCylinders(integer)`, and `numberOfVehicles(integer)` to keep track of all the vehicles created.

   Methods: suitable constructor(s), setters, getters, a `display()` method to show the vehicle's information, and a `getPrice()` method to calculate the total price of the vehicle after the taxes.

   - Design a class named "`Car`" which is a subclass of the `Vehicle` class and contains the following.

Data members: `numberOfPassengers(integer)`. The class also contains a `numberOfVehicles (integer)` to keep track of all the cars created.
Methods: suitable constructor(s), setters, getters, and override both the `display()` and `getPrice()` methods.
Taxes for cars are calculated according to the following rule:

$$\begin{cases} 15\%\ of\ \boldsymbol{price} & \boldsymbol{numberOfCylinders} \leq 4 \\ 30\%\ of\ \boldsymbol{price} & otherwise \end{cases}$$

- Design also a class named "`Truck`" which is a subclass of the `Vehicle` class and contains the following.
  Data members: `loadCapacity(integer)` in kilograms. The class also contains `numberOfVehicles(integer)` to keep track of all the trucks created.
  Methods: suitable constructor(s), setters, getters, and override both the `display()` and `getPrice()` methods.
  Taxes for trucks are calculated according to the following rule:

$$\begin{cases} 10\%\ of\ \boldsymbol{price} & \boldsymbol{numberOfCylinders} \leq 6\ and\ \boldsymbol{capacity} < 3500\ kg \\ 20\%\ of\ \boldsymbol{price} & otherwise \end{cases}$$

- Test the classes by creating an array of vehicles, some of which are created as `Car` objects and others as `Truck` objects. Find the average price of the total prices for each type of vehicle.